Waste Segregation Using Deep Learning

Submitted by:

Yalamanchili Vatsal-20233585
Tanishq Gupta-20233290
Tanishk Agrawal-20233289
Vaibhav Singh-20233300

Under the guidance of

Dr. Abhimanyu Sahu



COMPUTER SCIENCE AND ENGINEERING DEPARTMENT
MOTILAL NEHRU NATIONAL INSTITUTE OF TECHNOLOGY
ALLAHABAD, PRAYAGRAJ-211004, INDIA
October 29, 2026

Waste Segregation Using Deep Learning

1) Abstract

This project investigates image-based classification of waste into Biodegradable, Nonbiodegradable, and Recyclable categories. Its objective is to support segregation through learned visual features and an image-upload prediction interface. The implementation progresses from ResNet50 transfer learning and a custom residual-network experiment to EfficientNetB0 with two-stage training. The latest notebook records 42,007 readable images, an approximately 80:20 stratified training-validation split, and a final validation accuracy of **97.52%** with loss **0.0863**. Fine-tuning improves the best recorded accuracy from 92.84% for the frozen-backbone classifier to 97.52%. A Streamlit script implements inference using the selected V4 checkpoint.

These findings establish a functioning experimental classification pipeline, but do not establish independent real-world performance. Historical experiments differ in data composition, preprocessing, checkpoint selection, and validation procedures. No independent test set or V4 class-specific evaluation is saved. The principal contribution is the integrated data-loading, transfer-learning, checkpoint-selection, and inference workflow; its practical reliability requires further validation. [1-6]

2) Introduction

2.1 Background and problem statement

Waste segregation requires identifying materials before selecting an appropriate disposal or recovery stream. This project treats that task as supervised three-class image classification: an RGB image is mapped to one repository label. Image classification can assist this process, although appearance alone does not determine material composition, contamination, or local acceptance rules. [5]

2.2 Objectives and scope

The implemented objectives are to organize labeled waste images, address class imbalance, train and compare convolutional models, evaluate validation performance, and generate a category for an uploaded image. The scope is image-level classification rather than object detection, instance segmentation, mechanical sorting, or hazardous-material identification.

2.3 Repository structure and evidence

```
Waste-Segregation-Model/
│
├── DATASET/                    # Waste image dataset
│   ├── Biodegradable/          # Biodegradable waste images
│   ├── Nonbiodegradable/       # Nonbiodegradable waste images
│   └── Recyclable/             # Recyclable waste images
│
├── NoteBooks/                  # Model development and experiments
│   ├── model_1.ipynb           # Initial model
│   ├── version2_(1).ipynb      # Version 2
│   ├── version-03.ipynb        # Version 3
│   ├── version_4_latest.ipynb  # Latest V4 training pipeline
│   └── best_model_v4.keras     # Saved model
│
├── output/                     # Evaluation results and figures
├── report_work/                # Report preparation files
└── utils/                      # Supporting scripts and utilities
```

3) Methodology and Implementation

3.1 Dataset organization and preparation

The current DATASET directory contains 14,519 Biodegradable, 13,772 Nonbiodegradable, and 13,716 Recyclable files, totaling 42,007. These counts agree with V4's saved readable-image counts. Material examples include food and leaf waste, electronics and plastics, and cardboard and paper. Some data are nested further in folders such as default and real_world, which the V4 recursive loader includes.[1-5]

Table 1: Historical label counts and the latest saved V4 split. The CSV is an earlier manifest, not the V4 file list. [1, 2]

Class	Historical CSV	V4 train	V4 validation
Biodegradable	12,728	11,615	2,904
Nonbiodegradable	13,774	11,017	2,755
Recyclable	4,572	10,972	2,744
Total	31,074	33,604	8,403



Figure 1: Original class distribution from model_1.ipynb, cell 9. Recyclable images are substantially underrepresented in the historical 31,074-image manifest, motivating augmentation and weighted-loss experiments.

The baseline notebook generates two transformed copies per Recyclable image using rotation, shifts, shear, zoom, and horizontal flips. Its later manifest records 13,716 Recyclable images. V4 opens each supported JPG, JPEG, PNG, or BMP with Pillow, fully decodes it, converts it to RGB, and skips unreadable files without altering originals. It shuffles each class with NumPy seed 42, assigns floor(0.8N) images to training, and retains the remainder for validation. [1, 2]

3.2 Preprocessing and model architectures

All principal models use 224 x 224 RGB inputs. V4 uses bilinear resizing and float32 values in the 0-255 range, relying on EfficientNetB0's internal rescaling. Its tf.data pipeline maps a Pillow loader, shuffles training paths with a buffer of up to 4,096, batches 32 images, and prefetches automatically. Training-time augmentation applies horizontal flips, rotation 0.08, zoom 0.10, and contrast 0.10. The validation pipeline has no random augmentation. No handcrafted feature engineering or image normalization beyond the model pipeline is implemented. [1]

V4 uses an ImageNet-initialized EfficientNetB0 without its original classifier, followed by global average pooling, dropout 0.35, a 128-unit ReLU layer, dropout 0.25, and a three-unit softmax output. Earlier ResNet50 variants use global pooling and dense classification heads. The custom V3 defines residual blocks and additional hop additions; these are project-specific modifications, not a standard pretrained ResNet50 architecture. [1-4, 8, 9]

3.3 Training configuration

V4 first freezes EfficientNetB0 and trains the classification head for eight epochs using Adam at 0.001. It then reloads the best head checkpoint, enables the last 50 backbone layers except BatchNormalization layers, and fine-tunes for 20 epochs using Adam at 0.00001. The backbone is called with training=False to preserve inference-mode batch-normalization behavior. Both stages minimize sparse categorical cross-entropy. Checkpoints maximize validation accuracy; early stopping monitors validation loss with patience 5, and ReduceLROnPlateau halves the learning rate after two non-improving epochs, with a lower bound of 0.0000001. [1]

3.4 Training Workflow

The V4 model followed a two-stage transfer learning workflow using EfficientNetB0.
First, the pretrained backbone was frozen, and the classification layers were trained for eight epochs. Next, the best first-stage checkpoint was loaded, and the last 50 backbone layers were fine-tuned, with Batch Normalization layers kept frozen. A stratified validation set containing approximately 20% of each waste category was used to monitor performance after every epoch.Checkpointing tracked validation accuracy, while learning-rate reduction and early stopping monitored validation loss. The checkpoint with the highest validation accuracy across both gets saved. The selected model achieved 97.52% on the same validation set, representing a model-selection validation result rather than an independent test result.

3.5 Prediction Workflow

The demo caches the loaded checkpoint, converts an upload to RGB, resizes it identically to V4, predicts a batch of one, and selects argmax across the fixed class order. It displays all three softmax scores and explicitly notes that they are not necessarily calibrated probabilities. [5]

4) Results and Discussion

Table 2: Results as recorded in notebook outputs. Splits and evaluation protocols differ, so rows are not a controlled benchmark.

Experiment and evidence	Recorded validation result	Interpretation
Baseline ResNet50 [2], cell 25	53.06% peak; 45.45% evaluation	10-epoch run; augmented validation
Weighted ResNet50 V2 [3], cell 20	62.09% vs 58.68%	Separately loaded checkpoints
V3 custom hop training [4], cell 5	45.143% peak in saved log	Run output ends during epoch 19
V4 head / fine-tuned [1]	92.84% / 97.52%; loss 0.0863	Same V4 split; strongest paired evidence

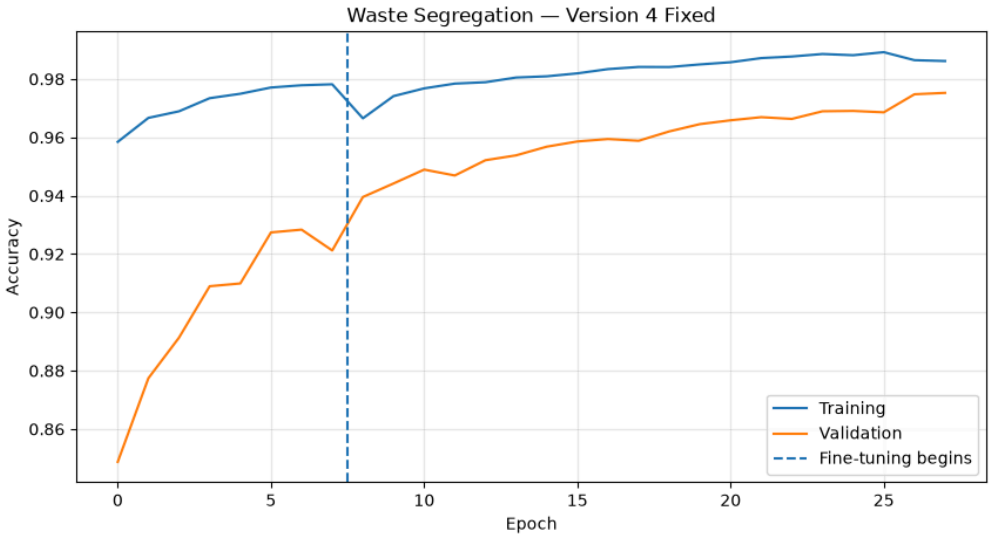


Figure 2: V4 saved training and validation accuracy over eight head-training and 20 fine-tuning epochs. The dashed boundary marks fine-tuning. Validation accuracy rises substantially while the gap to training accuracy narrows. [1, cell 0]

4.1 Comparison and class-specific behavior

The strongest internal comparison is V4's two stages: the checkpoint messages show 0.92836 for the best head model and 0.97525 for the best fine-tuned model, approximately **4.69 percentage points** apart using those printed values. The notebook summary rounds these to 92.84% and 97.52%. Final validation loss is 0.0863. The last logged training accuracy is 98.62%, versus validation accuracy 97.52%. This relatively small gap is encouraging, but is insufficient evidence of generalization to unseen collection sources. [1]

V4 prints a 35.43-point numerical difference from the V2 62.09% result. This is descriptive only: V2 uses 6,571 validation images, weighted loss, 0-1 input scaling, and a generator that also augments validation images. V4 uses 8,403 images and a randomized class-stratified split. The architecture, data, and evaluation changes prevent attributing that difference solely to EfficientNetB0.

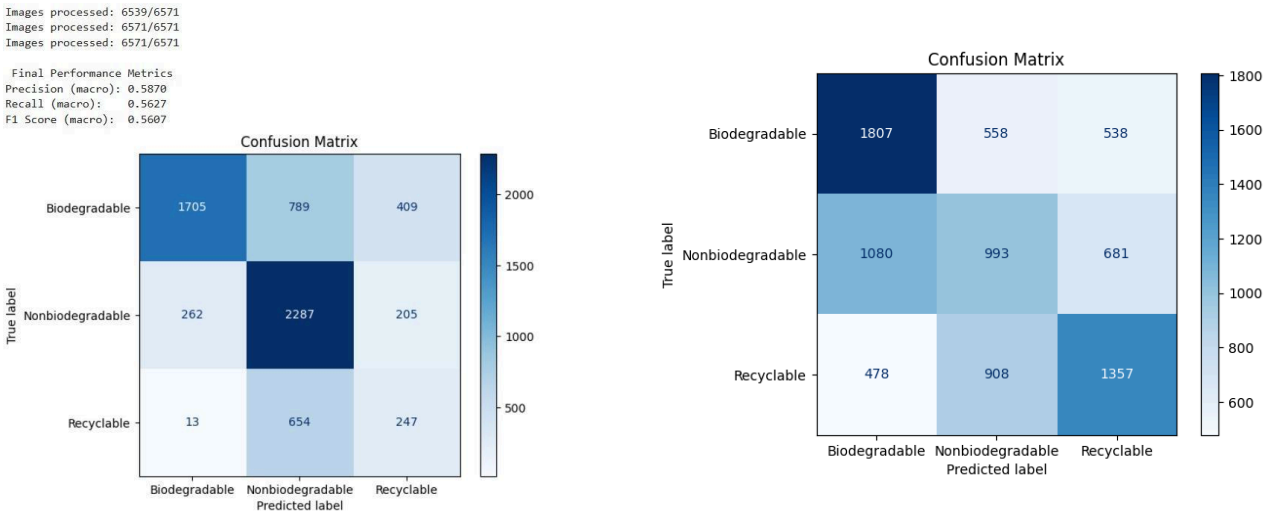
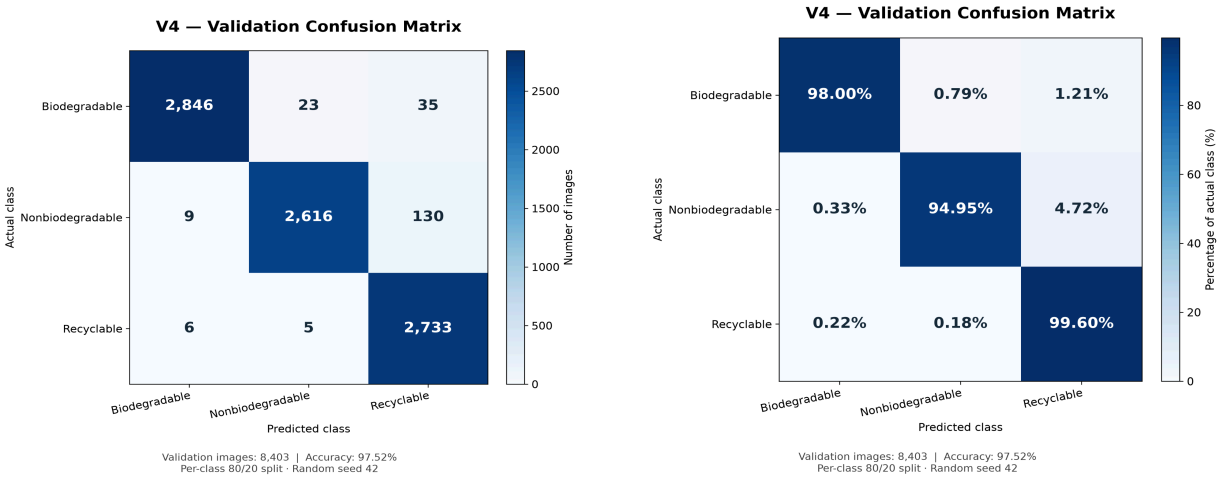


Figure 3 contains 6,571 observations. Its diagonal totals 4,239, giving **64.51% accuracy calculated from the displayed matrix**, distinct from the 62.09% checkpoint result. Only 247 of 914 Recyclable observations are correctly classified, demonstrating why overall accuracy should be accompanied by per-class recall. This screenshot supports class-level interpretation but does not document a fully traceable local training run for that checkpoint.

Table 3: Saved V3 classification report associated with Figure 4. Macro precision, recall, and F1 are each printed as 0.49; accuracy is printed as 0.49. [4, cell 8]

V3 class	Precision	Recall	F1	Support
Biodegradable	0.54	0.62	0.58	2,903
Nonbiodegradable	0.40	0.36	0.38	2,754
Recyclable	0.53	0.49	0.51	2,743

V3's prediction cell first loads an external best_model (4).keras, so Figure 4 and Table 3 cannot be attributed to the custom hop training run. Another attached V3 screenshot reports macro precision 0.5585, recall 0.5582, and F1 0.5561 with a different matrix. These are separate artifacts, not a single consistent evaluation. V4 has no saved confusion matrix, precision, recall, or F1, so its class-level weaknesses cannot be inferred from these earlier results.



The confusion matrix shows the performance of the V4 waste classification model on 8,403 validation images across three categories: Biodegradable, Nonbiodegradable, and Recyclable. The model correctly classified 2,846 biodegradable, 2,616 nonbiodegradable, and 2,733 recyclable images, achieving an overall validation accuracy of 97.52%. The diagonal entries represent correct predictions, while the off-diagonal entries indicate misclassifications. The most frequent error was the classification of 130 nonbiodegradable images as recyclable, suggesting that distinguishing between these two categories remains the model's main challenge. Overall, the results demonstrate strong classification performance across all three waste categories on the validation dataset.

5) Conclusion and Future Work

The project implements a complete experimental workflow from labeled waste images to trained convolutional classifiers and a Streamlit upload interface. V4 provides the strongest recorded validation result at 97.52% accuracy and 0.0863 loss, with a clear improvement after fine-tuning on the same split. The available evidence supports successful model development and inference implementation, while operational sorting accuracy remains unverified. Historical comparisons also show that preprocessing, validation design, and checkpoint identity materially affect interpretation.

Future work should establish image-source provenance, remove duplicates, group original images with their augmentations before splitting, and reserve an untouched test set. All models should then use identical partitions and documented preprocessing. V4 needs per-class precision, recall, F1, confusion matrices, and representative error inspection. Further priorities are repeated runs, calibration and abstention for uncertain inputs, a reproducible environment, latency tests, and a reviewed label taxonomy that distinguishes material identity from local disposal policy.

References

[1] Project artifact: NoteBooks/version_4_latest.ipynb, cell 0, source and saved outputs; V4 checkpoint configuration and metadata.

[2] Project artifacts: NoteBooks/model_1.ipynb, cells 4-27; utils/dataset_labels.csv; DATASET folder inventory.

[3] Project artifact: NoteBooks/version2_(1).ipynb, cells 8-22, including embedded evaluation screenshot.

[4] Project artifact: NoteBooks/version-03.ipynb, cells 1-10, including saved output and attachments.

[5] Project artifacts: README.md and waste_demo.py.

[6] LICENSE: MIT License, copyright 2026 Tanishq Gupta.

[7] Tan, M. and Le, Q. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. ICML, PMLR 97, 6105-6114. https://proceedings.mlr.press/v97/tan19a.html

[8] He, K., Zhang, X., Ren, S. and Sun, J. (2016). Deep Residual Learning for Image Recognition. CVPR, 770-778. https://arxiv.org/abs/1512.03385